OutOfRole: Plan, Changes and Findings

The Debugging Table · IBM Z Datathon 2026 · AI Secured: Innovation Without Exposure

October 16, 2026 (revised after external review)

Purpose

This document records what we will build on October 17 and 18, why the plan changed from our abstract, and what each step of our pre-event study found. The full study, with code and outputs, is in `OutOfRole_Feasibility.ipynb` (same folder). It has been through **three rounds of external review** (Codex, gpt-6.1-sol); the reviews are in `codex_review/`. Every number below comes from the notebook's final run.

The question: can we rank employees so that people misusing legitimate access (insiders) rise to the top, using only activity logs and no labels? Does a recommender system ("access that is surprising for someone like you") add anything over simpler signals?

Data: CMU SEI CERT Insider Threat Test Dataset.

- **r4.2:** 1,000 staff, 70 insiders, three attack types: leakers, job-hunter thieves and disgruntled sysadmins.
- **r5.2:** 2,000 staff, 99 insiders. It adds a fourth type: snoopers who log into colleagues' computers.

We train only on the months before any attack (r4.2 until June 1, 2010; r5.2 until April 1, 2010), drop the week that contains each cutoff, score every person once, and use the answer key only to grade. Top-k counts are expected values with tied scores ordered at random, so some are fractional.

The plan for the build

Stage	What	Status
1. Load	PySpark in local mode: daily, person-to-computer and person-to-site tables (install Java 17 and PySpark; no pyarrow on s390x)	tested
2. Baselines	Personal daily and 28-day baselines, shown as the bar to beat	tested
3. Main ranked list	**Frozen family-first Stouffer fusion** of signal families: behavior windows, own-history weekly percentiles, team peers, rare sites, weekly LOF and autoencoder, recommender. Baseline and percentile lists shown alongside for comparison	tested on both releases; beat the baseline at 100 reviews on both (55 vs 44; 43 vs 39)
4. Out-of-role view	Weekly persistence of logins to unusual computers, discounted for people who already roamed in training; shown as an **experimental investigation view** with no reserved review slots	r5.2 only (27 to 30 of 30 snoopers); designed after seeing r5.2
5. Evidence	Per-person facts for Granite: signal contributions and timelines; the recommender's "nobody like you visits this site" as an optional fact	recommender kept inside the tested fusion; its individual necessity not proven
6. Explain and show	Granite cited explanations; dashboard; honest evaluation table (baseline vs system, per attack type, unseen-attack test, equal review budget)	not yet built; Granite reliability and 3-vCPU runtime untested

The combination of stages 3 and 4 was not tested as a unit. Freeze its settings before evaluating it.

What changed from the abstract, and why

Original plan	Now	Reason (study step)
Recommender (ALS) as the core engine	Optional evidence inside the fused ranking; not a queue	At its own job it is at the popularity baseline; as a queue it displaced more insiders than it added (Step 15)
"The recommender forgives colleagues' computers; an ownership rule fixes it"	Snoopers are caught by **persistence** (weekly counting plus a roaming discount); ownership only sharpens the top of the list	With identical counting, ownership, naive and recommender versions each catch 27 of 30 (Step 13)
Four specialist queues	Fused ranking as the default, plus an experimental out-of-role view	At 100 reviews, fusion won on r4.2 (55 vs 42) and queues won on r5.2 (53 vs 43). Neither won on both, and fusion beat the baseline on both (Step 14)
Web log as a stretch goal	Web log is core	Rare-site visits are a strong signal; Spark processes the log in under a minute (Steps 1, 4)
Trained combiner as the headline	Label-free signals as the headline	With job-hunters hidden from training, trained models catch 4 to 7 of 30; the label-free sum catches 16 (Step 7)

Results at a glance

"found" = expected insiders in the top 70 (of 70 in r4.2, of 99 in r5.2).

Approach	r4.2: P@20 / found / AP	r5.2: P@20 / found / AP	Unseen attack type
Baseline: 28-day personal window	1.00 / 38 / 0.728	0.90 / 36 / 0.428	catches spike attacks only
Own-history weekly percentile	1.00 / 39.4 / 0.666	1.00 / 40.3 / 0.477	best top of the list on both releases
Label-free family-first Stouffer (frozen on r4.2)	1.00 / 45 / 0.784	0.90 / 38 / 0.478	no attack-label fitting
Trained combiner (logistic regression)	1.00 / 52 / 0.857	0.90 / 49 / 0.630	hidden job-hunters 5 to 16 of 30; r5.2 hidden leakers 0 to 28 of 29
Gradient boosting	1.00 / 60 / 0.940	not run	hidden job-hunters 4 of 30
Weekly persistence, unusual computers	no snoopers in r4.2	27 to 30 of 30 snoopers in the top 100	designed after seeing r5.2

Equal budget, exactly 100 people reviewed (insiders found):

Method	r4.2	r5.2
Four queues	42	53 (59 without the recommender queue)
Baseline R2	44	39
Own-history percentile	43	44.6
Frozen family-first Stouffer	**55**	43
Trained combiner (needs labels)	63	55

What the recommender contributes (Step 15):

- **Its own job:** predicting which new sites normal staff visit, its mean percentile rank is 35.9% vs 35.5% for "guess the popular sites" on r4.2, and 35.8% vs 38.3% on r5.2 (50% = random). RMSE is worse than popularity because implicit-feedback scores aren't calibrated, so RMSE is a diagnostic only.

- **Detection:** adding recommender features to naive novelty in the combined model gives r4.2 51 to 52 and r5.2 45 to 49 insiders in the top 70. As a review queue on r5.2 it found 5 insiders the other queues missed, but displaced 11.
- **Verdict:** modest, conditional value; kept as optional evidence and a tested research branch.

Metrics:

- **P@20:** share of the top 20 who are real insiders.
- **AP (average precision):** summarizes the whole ranking.
- **AUC:** the chance a random insider ranks above a random normal employee.
- **Unseen attack (leave one scenario out):** a combiner is trained without one attack type, then tested on it.

Findings at each step

Step	Why we did it	Key result	What changed
1. Load with Spark	15 GB (r4.2) and 37 GB (r5.2) of logs	About 36 s on a cold laptop run; 1,003 computers and 475 websites	Web log becomes core
2. Leakage check	Reviewer: time splits can leak answers	No attack before the cutoffs; boundary weeks dropped. All 70 r4.2 insiders leave; in r5.2 the 29 who stay are snoopers	Never use departure data
3. Baselines	A model must beat simple rules	Personal baselines: P@20 1.00, 38 of 70, AUC up to 0.96; only 2 to 5 of 30 job-hunters	Baselines are the bar
4. Novelty	The project's core idea	Rare-site visits: 41 of 70. Recommender website surprise: AUC 0.71 vs 0.59 for naive, but P@20 0.00	Test the recommender as an addition
5. First combinations	Reviewer: prove the recommender adds value	Learned combiner: 39, then 41 with naive novelty, then 45 with the recommender (45 and 44 on other seeds)	Keep it as one signal
6. Peers, windows, second model	Reviewer: covering colleagues, team-wide changes	Logistic regression 45 to 49 with ALS; gradient boosting 60 of 70	Stress test needed
7. Unseen-attack test (repaired)	Real attackers invent new methods	Hidden job-hunters: boosting 4, logistic 5 to 7, label-free sum **16** of 30	Don't headline trained combiners
8. Literature models	Published CERT work	**Weekly LOF finds 12 to 13 of 30 job-hunters**	Weekly LOF is the slow-drift signal
9. Recommender tweaks	Menon 2014, Google Facade, Microsoft	Website peer-of-item AUC 0.84 (17 of 70); lift and site-lift failed	Peer-of-item stays a candidate
10. Own-history percentiles	Le & Zincir-Heywood 2021	P@20 1.00, 39.4 of 70; combiner 47 to 51; hidden job-hunters 8 to 12	Core detector
11. Label-free fusion	Labels are rare in practice	Family-first Stouffer: P@20 1.00, 45 of 70, AP 0.784; 11 of 11 corroborated people were insiders	Frozen for the r5.2 test
12. r5.2, pipeline frozen	Guard against tuning to r4.2	Own-history percentile best (AP 0.477); frozen fusion beats the baseline (AP 0.478 vs 0.428); trained combiners swing widely on unseen attacks	Fusion survives as the default
13. Out-of-role signal	Snoopers need their own signal	Ownership rule: 30 of 30. Controlled: ownership, naive and recommender versions all 27 of 30	Persistence, not ownership or ALS
14. Queues vs fusion	Averaging can hide specialists	At 100 reviews: fusion 55 vs queues 42 (r4.2); queues 53 vs fusion 43 (r5.2)	Fusion default plus experimental view
15. Recommender contribution	"Does it help?"	At the popularity baseline on its own job; +1 to +4 in combiners; its queue cost 6 insiders on r5.2	Optional evidence only

What the external review changed

Codex reviewed the notebook three times. Every point was checked against the code and the data before acting on it. It found real bugs:

- The hold-one-attack-out test averaged held-out insiders over 25 models and normal staff over 5.
- The week containing each cutoff leaked six test days into training. No attack days were involved.
- Top-k counts depended on user-ID order when scores tied.
- An equal-budget comparison silently used a different fusion method.

All were fixed and the notebook was re-run. The fixes overturned three headline claims: "the recommender adds 7 insiders", "collaborative filtering forgives colleagues' computers", and "four queues are best". Saying this openly is a strength in front of judges.

Honest limits to state in the pitch

- **Retrospective, not real-time:** each person is scored over the whole test period; detection delay is not measured.
- **Job-hunters stay hard:** 0 to 16 of 30 in the top 100, depending on the method. Published work reports 12.5 to 74%.
- **The data is synthetic:** it's the standard benchmark. Labeled real insider logs are very rare; LANL releases real authentication logs with red-team labels, which would make a good third test.
- **Partly post-hoc design:** the out-of-role view and this build plan were shaped after seeing r5.2, and need an untouched test.

Related documents

- `OutOfRole_Feasibility.ipynb` (and `.html`): the full study with code, outputs and reviewer questions.
- `codex_review/`: the three external review rounds and our response.
- `../Research_Findings.md`: published work, industry practice, LinuxONE notes and sources.
- `../output/IBM_Z_Datathon_2026_OutOfRole_Team_Guide.pdf`: VM, SSH, Docker, Jupyter and PySpark setup. Its Chapter 9 ("Implement OutOfRole") is superseded by this plan.
- `../Data_Access_Guide.pdf`: how each teammate downloads the data.